Project Aurora-927 — Security & Deployment Assessment

Technical Engineering Report

DRAFT - For engineering review; site verification required

Generated 05 November 2026 (UTC)

Contents

1. Executive Summary 3

2. Scope, Assumptions, and Assessment Limitations 3

3. System Context and Technology Overview 4

4. Authentication Flow and Keycloak Integration 5

5. JWT Validation and Token Handling 6

6. API Access Controls and Authorization Model 7

7. Database Security and PostgreSQL Protection 7

8. Deployment Architecture and Trust Boundaries 8

9. Risk Assessment Table: Hypothetical Risks and Proposed Mitigations 10

10. Detailed Security Risks and Mitigations 11

11. Deployment and Release Checklist 12

12. Monitoring, Logging, Incident Response, and Maintenance 12

13. Final Recommendations and Technical Review Priorities 13

14. Appendices: Glossary, Assessment Basis, and Page Numbering 14

1. Executive Summary

Project Aurora-927 is assessed as a fictional web application comprising a React front end, a Django-based API, Keycloak for identity and access management, and PostgreSQL for persistent data storage. This assessment provides a structured technical review of the application's anticipated security and deployment posture, with emphasis on authentication, token validation, authorization, database protection, deployment boundaries, operational controls, and release readiness.

The proposed architecture can provide a sound security foundation when responsibilities are clearly separated and consistently enforced. Keycloak should act as the authoritative identity provider, while Django remains responsible for validating issued tokens and applying application-specific authorization decisions. The React client should be treated as an untrusted public interface: it may initiate authenticated requests, but it must not be relied upon to enforce permissions or protect sensitive data. PostgreSQL should remain isolated from public network access and accessible only through appropriately controlled application services or administrative paths.

This report distinguishes design considerations and hypothetical risks from confirmed vulnerabilities. No penetration testing, source-code review, configuration inspection, operational observation, or measured security testing is represented in this document. Consequently, the identified risks are proposed review items rather than verified defects. Their applicability depends on the final implementation, deployment configuration, Keycloak realm settings, network controls, secrets-management practices, and operational procedures.

The principal technical priorities are to establish a precise trust model, validate JWTs using issuer and audience checks in addition to signature verification, enforce authorization on every protected API operation, and prevent client-controlled claims or identifiers from determining access. Deployment should use encrypted communications, restrictive network segmentation, managed secrets, hardened container or host configurations, and controlled administrative interfaces. Database protection should include least-privilege accounts, encrypted connections where appropriate, secure backups, and careful handling of sensitive records.

Before release, the project team should confirm the complete authentication and authorization flow, document token lifetimes and key-rotation behavior, verify failure handling, and review logging for both security value and privacy impact. The deployment checklist should also address dependency management, security headers, cross-origin policies, environment separation, backup restoration, monitoring, incident response, and rollback procedures.

Overall, Aurora-927 appears capable of supporting a defensible security architecture, provided that security controls are implemented centrally, tested against defined requirements, and maintained throughout the application lifecycle. Technical review should prioritize authorization correctness, identity-provider integration, secret and key management, exposure of deployment services, and the ability to detect and respond to misuse.

2. Scope, Assumptions, and Assessment Limitations

This assessment covers the proposed security and deployment architecture for Project Aurora-927, a fictional application comprising a React front end, Django-based API services, Keycloak for identity and access management, and PostgreSQL for persistent data storage. The review addresses the principal interactions among these components, including user authentication, token issuance and validation, API authorization, database access, deployment boundaries, operational controls, and release readiness. It also identifies hypothetical risks and proposes mitigations suitable for technical review and subsequent validation.

The assessment is architectural and documentation-oriented. It is intended to support design decisions, implementation planning, and deployment preparation rather than to certify a production environment. No claim is made that the application has been penetration tested, scanned, code reviewed, load tested, or operated under representative production conditions. Unless explicitly stated elsewhere, identified concerns should be treated as

potential risks or control gaps requiring verification, not as confirmed vulnerabilities.

The assessment assumes that the React application is delivered to users through a controlled hosting or content-delivery environment, that Django exposes authenticated API endpoints, that Keycloak acts as the authoritative identity provider, and that PostgreSQL is not directly accessible from untrusted networks. It further assumes that deployment environments, secrets, certificates, configuration, and operational ownership are managed separately from application source code. These assumptions establish a reasonable basis for analysis but must be confirmed against the actual implementation and infrastructure.

The review does not prescribe a specific cloud provider, container platform, network topology, Keycloak version, Django version, PostgreSQL version, hosting model, or CI/CD product. Consequently, provider-specific security features, version-specific defects, licensing considerations, and detailed configuration requirements remain outside the available assessment basis. The report also does not establish regulatory compliance, privacy-law conformity, business continuity objectives, or a formal threat model unless those requirements are separately defined by the project stakeholders.

Several limitations affect the conclusions. The fictional nature of the project means that no source repository, deployment manifests, infrastructure diagrams, runtime configuration, logs, dependency inventory, database schema, or test evidence is available for direct examination. Security properties such as issuer validation, audience enforcement, key rotation, role mapping, TLS configuration, network isolation, backup protection, and audit logging therefore require implementation-level confirmation. Similarly, the proposed risks are scenario-based and are not ranked using observed exploitability or measured operational impact.

Recommendations should be validated through secure configuration review, automated and manual testing, deployment rehearsals, and approval by the system owner and relevant security stakeholders. Any change to architecture, identity flows, data sensitivity, or operational responsibility should trigger a corresponding reassessment of the assumptions and limitations described here.

3. System Context and Technology Overview

Project Aurora-927 is assessed as a fictional web application composed of a React-based user interface, a Django backend, Keycloak for identity and access management, and PostgreSQL for persistent data storage. The application is assumed to expose authenticated business functions through HTTP-based APIs. The principal security objective is to ensure that users and services are authenticated consistently, access decisions are enforced at the API boundary, sensitive data is protected in transit and at rest, and operational components remain separated according to their responsibilities.

The React frontend is responsible for presentation, client-side interaction, and initiating requests to the Django API. It should be treated as an untrusted execution environment because code, configuration, and browser-held state may be inspected or modified by an end user. The frontend must therefore not be relied upon as the authoritative enforcement point for permissions, data filtering, or tenant isolation. Any controls implemented in the interface should improve usability, but equivalent authorization decisions must be independently enforced by Django for every protected operation.

Django functions as the application and API tier. It is expected to validate incoming requests, establish the authenticated principal from a bearer token or other approved mechanism, apply endpoint and object-level authorization, validate input, and mediate all access to PostgreSQL. Security-sensitive behavior should be centralized through well-defined middleware, authentication components, serializers, views, and permission policies rather than implemented inconsistently across individual endpoints. Administrative interfaces, diagnostic routes, and undocumented API operations require the same protection and deployment review as user-facing functionality.

Keycloak provides the identity layer and is responsible for user authentication, token issuance, realm configuration, client registration, role or group representation, and related identity lifecycle functions. Its configuration forms part of the application's security boundary. Issuer values, approved audiences, signing algorithms, key rotation behavior, redirect URI restrictions, client confidentiality, and administrative access must be governed explicitly. Keycloak should be integrated with Django through standards-based token validation and should not be treated merely as a login screen.

PostgreSQL is the system of record and should be reachable only by authorized application components over controlled network paths. Database credentials should be managed as deployment secrets, assigned according to least privilege, and separated by environment where practical. Application-level authorization remains necessary even when database connectivity is restricted, because network isolation alone does not determine whether a user may access a particular record.

The deployment context should include protected transport between users, the frontend, Django, Keycloak, and PostgreSQL, together with controlled configuration management, logging, backups, and administrative access. The specific infrastructure topology is not established by the available description; consequently, this section defines the intended technology roles and trust relationships rather than confirming implementation details or reporting test findings.

4. Authentication Flow and Keycloak Integration

Project Aurora-927 uses Keycloak as the central identity provider for authentication, with the React frontend acting as the client application and Django serving as the protected API. This arrangement separates identity management from application business logic and allows authentication policies, user lifecycle controls, and token issuance to be managed consistently. The precise Keycloak realm, client identifiers, redirect URIs, and deployment settings are implementation-dependent and should be verified against the deployed configuration.

A typical browser-based flow begins when an unauthenticated user accesses a protected frontend route. The React application redirects the user to Keycloak using an authorization-code flow with Proof Key for Code Exchange (PKCE), where supported by the client architecture. Keycloak authenticates the user and applies configured controls such as password policy, multi-factor authentication, account status checks, and identity-provider federation. Following successful authentication, Keycloak returns an authorization code to the registered redirect URI. The frontend exchanges that code through the approved protocol flow and receives tokens appropriate to the configured client.

The access token is presented by the frontend when calling Django API endpoints, normally in the HTTP Authorization header using the Bearer scheme. Django should treat Keycloak as the issuer of identity assertions but should not rely on frontend state, decoded browser data, or user-supplied identity fields. Each API request must be independently evaluated using the token's cryptographic signature, issuer, audience or authorized-party claims, expiration, and other applicable validation rules. Detailed token-processing controls are addressed in Section 5.

Keycloak client configuration is a significant security boundary. Redirect URIs should be explicit and limited to approved application origins; broad wildcard patterns can enable authorization-code interception or token delivery to an unintended location. Client types and authentication methods should match the application design. Public browser clients must not contain a client secret, while confidential clients require protected secret storage and controlled rotation. Development, test, and production realms or clients should be separated to prevent credentials, users, roles, and redirect settings from being inadvertently reused across environments.

Logout and session handling should be defined across both systems. The frontend should remove locally retained tokens through a controlled mechanism, while Keycloak logout or session termination should be used where centralized invalidation is required. Token storage warrants particular care: long-lived tokens in browser-accessible

storage may increase the impact of cross-site scripting, whereas in-memory handling or secure, appropriately scoped cookies may reduce exposure depending on the architecture. Cross-origin resource sharing, redirect validation, and anti-forgery protections must be aligned with the selected token and session model.

Because this assessment is based on the stated fictional architecture rather than verified configuration or testing, weaknesses in the described flow are hypothetical until confirmed through configuration review, code inspection, and controlled security testing.

5. JWT Validation and Token Handling

The application should treat every JSON Web Token (JWT) presented to a Django API as untrusted input until it has been cryptographically and semantically validated. Validation should be performed by a maintained library configured for the Keycloak realm that issued the token. The API should obtain trusted signing keys from the realm's published JWKS endpoint, cache them for an appropriate period, and refresh them when an unfamiliar key identifier (kid) is encountered. The accepted signing algorithm must be explicitly configured; the application should not infer trust from the token header or accept an unsigned token.

Validation must include the token signature and the principal claims required by the deployment design. In particular, the API should verify the expected issuer (iss), audience (aud), expiration (exp), and, where applicable, not-before (nbf) and issued-at (iat) values. Audience checking is important when a Keycloak realm issues tokens for multiple clients, because a valid token intended for another service should not automatically authorize access to this API. Small, documented clock tolerance may be permitted to accommodate time synchronization differences, but excessive tolerance would extend the effective lifetime of expired or not-yet-valid tokens. Host and container clocks should therefore be synchronized through the platform's approved time service.

After cryptographic validation, the API should map claims to an application identity and authorization context. The mapping should define whether access is based on realm roles, client roles, groups, scopes, or a combination of these claims. Claim names and nesting should be treated as configuration rather than assumed, particularly when Keycloak client settings or protocol mappers may change. Missing, malformed, or ambiguous authorization claims should result in denial rather than an implicit default role. Authentication middleware should establish the validated subject and relevant claims once per request, while endpoint-level authorization should make the required permission explicit.

Bearer tokens should be accepted only over authenticated TLS connections and should not be placed in URLs, query strings, or browser history. For a React client, the preferred storage and renewal approach should minimize exposure to cross-site scripting; long-lived tokens in persistent browser storage present greater risk than short-lived access tokens held in memory with a carefully designed renewal flow. Refresh tokens should be used only with the designated authorization flow, protected as sensitive credentials, and revoked or invalidated when sessions are terminated or compromise is suspected. The API should not accept refresh tokens in place of access tokens.

Validation failures should produce consistent, non-sensitive responses, such as an unauthorized result, without revealing whether a token failed signature, issuer, audience, or expiry checks. Logs may record a correlation identifier and a sanitized failure category, but must not record complete tokens or sensitive claim sets. Key rotation, token lifetime, revocation behavior, and emergency issuer or key changes should be documented and exercised through operational procedures.

6. API Access Controls and Authorization Model

The API should enforce authorization independently of the client application and should not treat successful authentication as evidence that a caller may access every resource. Each protected endpoint should apply a default-deny policy and require an explicit permission decision based on the validated JWT, the requested operation, and the target resource. React components may hide unavailable functions for usability, but those interface controls must not be relied upon as security boundaries.

Keycloak should provide the authoritative identity and role context, while Django should translate that context into application permissions. Realm roles, client roles, and, where appropriate, token scopes should be mapped to narrowly defined API capabilities such as read, create, update, delete, or administrative actions. The mapping should be documented and stable, with ambiguous, missing, or unrecognized claims resulting in denial rather than implicit access. Administrative privileges should be separated from ordinary user privileges, and roles should follow least-privilege principles.

Django REST Framework permission classes or an equivalent centralized authorization layer should be used consistently across viewsets and individual endpoints. Authorization checks should cover both the HTTP method and the business action. For example, a user permitted to view a project should not automatically be permitted to modify membership, export data, or change security settings. Sensitive operations should require an explicit elevated permission and, where justified by risk, additional confirmation or reauthentication through the identity provider.

Object-level authorization is essential wherever records belong to a user, organization, project, or other tenant. Querysets should be constrained to resources within the caller’s authorized security context before records are returned or modified. Direct lookup by an identifier must not bypass these constraints. Update and delete operations should repeat the authorization decision at the point of action, reducing the risk that an object reference, altered request parameter, or stale client state exposes another tenant’s data. Bulk operations require equivalent per-object or policy-based controls.

Service-to-service access should use separate confidential clients or service accounts with narrowly scoped credentials and permissions. Application components should not share human user roles or use broad administrative tokens for routine processing. Token audiences, issuers, scopes, and expiration should be validated as described in the token-handling controls, and authorization should reject tokens intended for another service or API.

The authorization model should also define treatment of unauthenticated endpoints, health checks, documentation, file uploads, and error responses. Public endpoints should be intentionally listed and reviewed. Authorization failures should return consistent status codes without disclosing whether protected objects exist. Audit logging should record the principal, action, resource context, decision, and correlation identifier for sensitive authorization events, while avoiding unnecessary token or personal-data exposure. Because this assessment is design-focused, these controls represent required implementation and review criteria rather than confirmed findings.

7. Database Security and PostgreSQL Protection

PostgreSQL should be treated as a private, high-value trust-zone component rather than an application-facing service. The database should accept connections only from approved Django application instances or controlled administrative paths, using network policies, security groups, firewall rules, or equivalent controls. Public exposure of the PostgreSQL listener should be prohibited. Administrative access should use a separate management route with strong authentication, auditability, and narrowly defined source restrictions.

Database credentials should not be embedded in source code, container images, deployment manifests, or client-side configuration. Runtime secrets should be supplied through an appropriately protected

secrets-management mechanism, with access limited to the application workload and authorized operators. Credentials should be unique to each environment and, where practical, separated by function. The Django application should use a dedicated database role with only the permissions required for normal operation. Schema-management privileges should be assigned separately from routine read/write access, reducing the impact of application compromise.

Transport encryption should be enabled for connections between Django and PostgreSQL, particularly where traffic crosses hosts, clusters, or other infrastructure boundaries. Certificate validation should be configured so that encryption does not merely conceal an unauthenticated connection. PostgreSQL authentication rules should use explicit host-based access controls, approved authentication methods, and restrictive address ranges. Default accounts, unnecessary extensions, unused databases, and obsolete roles should be removed or disabled in accordance with the deployment model.

The application should rely on Django’s parameterized query mechanisms and ORM protections, while treating raw SQL as a controlled exception requiring review. Input validation remains necessary, but it should not be considered a substitute for safe query construction. Database errors returned to clients should be generalized to avoid exposing schema names, query fragments, credentials, or infrastructure details. Separate database roles and schemas may be appropriate where administrative functions, background jobs, and API traffic have materially different privileges.

Data protection should address both confidentiality and recoverability. Sensitive fields should be identified and protected according to their business impact, with encryption or application-level protection considered where database compromise would otherwise expose unacceptable information. Backups should be encrypted, access-controlled, monitored, and tested through restoration exercises. Retention periods should reflect operational and legal requirements, and backup copies should not be reachable using the same credentials as the production database.

Logging should capture authentication failures, authorization changes, privileged operations, schema modifications, and relevant connection events without recording passwords, tokens, or unnecessary sensitive values. PostgreSQL, operating-system, and application logs should be time-synchronized and retained according to the incident-response plan. These controls are recommended protections and assessment considerations; no specific PostgreSQL vulnerability or control failure is asserted without direct configuration evidence.

8. Deployment Architecture and Trust Boundaries

Project Aurora-927 should be deployed as a set of logically separated services rather than as a single trusted application environment. The React frontend is delivered to users through a browser and should be treated as an untrusted client. It may be hosted through a content delivery network or web server, but neither the frontend code nor values embedded in it should contain confidential credentials, signing keys, database connection strings, or administrative secrets. All security-sensitive decisions must be enforced by the backend and identity infrastructure.

The primary external trust boundary is the edge of the deployment, typically represented by a DNS endpoint, load balancer, reverse proxy, or ingress controller. This boundary should provide TLS termination with modern protocol settings, route only approved paths, and apply appropriate request-size, rate, and timeout controls. If traffic is forwarded to internal services, the forwarding layer should preserve reliable identity and request metadata without allowing clients to spoof trusted headers. Administrative endpoints and management interfaces should not be exposed through the same public route as ordinary application traffic unless access is explicitly restricted.

The Django application forms a separate application trust zone behind the edge. It should accept requests only through approved network paths and should validate authentication tokens, issuer and audience claims, signature algorithms, expiration, and required authorization claims before performing protected operations. Network placement is not a substitute for authorization: an internal service or a request originating from a permitted subnet

must not automatically be considered trusted. Service-to-service communication should use authenticated and encrypted channels where practical, particularly when crossing deployment zones or infrastructure boundaries.

Keycloak constitutes the identity trust boundary. The application may rely on Keycloak for token issuance and identity claims, but it should not grant broader permissions solely because a user is authenticated by that system. Keycloak administration, realm configuration, client registration, signing-key management, and user-management functions should be isolated from ordinary application traffic and limited to designated administrators. Publicly reachable identity endpoints should expose only the functions required for the selected authentication flows.

PostgreSQL should reside in a private data zone with no direct public exposure. Only the Django service, approved migration or maintenance processes, and narrowly defined administrative paths should be able to connect. Database credentials should be supplied through a managed secrets mechanism or protected runtime configuration, with separate credentials and privileges for application use, migrations, and administration. The application account should not possess unnecessary schema-owner or superuser capabilities.

Secrets, signing material, and configuration should be managed separately from container images and source repositories. Deployment pipelines, registries, orchestration platforms, backups, and operational consoles are additional trust boundaries requiring access control, audit logging, and least privilege. Because the exact hosting topology is not specified, these controls are deployment requirements and architectural expectations rather than confirmed properties of the current environment.

9. Risk Assessment Table: Hypothetical Risks and Proposed Mitigations

This assessment identifies eight hypothetical risks for the Aurora-927 application. They are based on the stated React, Django, Keycloak, and PostgreSQL architecture and are not confirmed vulnerabilities. Likelihood and impact are qualitative planning judgments; they should be validated through configuration review, testing, and operational evidence before release.

ID	Hypothetical risk	Likelihood	Impact	Proposed mitigation
R1	JWTs may be accepted without complete validation of signature, issuer, audience, algorithm, expiration, or key rotation state.	Medium	High	Centralize token validation in Django, explicitly configure accepted algorithms and claims, reject expired or malformed tokens, retrieve signing keys through a controlled Keycloak configuration, and test key-rotation behavior.
R2	Authorization checks may verify authentication but fail to enforce object-level or role-based access restrictions.	Medium	High	Define an authorization matrix for endpoints and resources, enforce permissions server-side, apply object-level ownership or tenancy checks, and include negative tests for horizontal and vertical privilege escalation.
R3	Keycloak configuration or administrative interfaces may be exposed beyond trusted management paths.	Low–Medium	High	Restrict administrative access through network controls and strong administrator authentication, minimize realm privileges, review clients and redirect URIs, and monitor administrative events.
R4	Secrets, service credentials, or signing-related configuration may be exposed through source control, images, logs, or deployment manifests.	Medium	High	Store secrets in an approved secrets manager, use environment-specific credentials, prevent secret logging, scan repositories and images, rotate credentials, and apply least privilege to service accounts.
R5	API endpoints may be vulnerable to injection, unsafe input handling, or excessive request payloads.	Medium	High	Use Django ORM parameterization, validate and constrain input at serializers and business-logic boundaries, limit request sizes and query complexity, and perform focused security testing of high-risk endpoints.
R6	PostgreSQL may be reachable from unnecessary network locations or operate with excessive database privileges.	Low–Medium	High	Place the database on a private network, restrict inbound rules, use TLS where appropriate, separate application and administrative roles, limit schema privileges, and protect backups with equivalent access controls.
R7	Cross-site request forgery, cross-origin misconfiguration, or insecure browser token handling may enable unauthorized actions or token exposure.	Medium	High	Configure trusted origins narrowly, use secure cookie attributes where cookies are used, apply CSRF protections to state-changing requests, avoid unnecessary browser token persistence, and establish a restrictive content security policy.
R8	Insufficient logging, alerting, or recovery procedures may delay detection and increase the effect of an incident.	Medium	Medium–High	Record authentication, authorization, administrative, and deployment events without sensitive token content; centralize protected logs, define alert thresholds, test backup restoration, and maintain an incident-response procedure.

Risk treatment should prioritize R1, R2, R4, and R6 because failures in these areas could directly affect identity trust, data confidentiality, or authorization boundaries. Residual risk should be documented after mitigation, including the responsible owner, target date, validation method, and any accepted exceptions.

10. Detailed Security Risks and Mitigations

The risks described in this section are hypothetical security concerns derived from the stated React, Django, Keycloak, and PostgreSQL architecture. They are not confirmed vulnerabilities, because no source code, configuration review, penetration test, or operational evidence has been provided. Each risk should therefore be validated during implementation and deployment review.

A primary risk is improper JWT validation by the Django API. If the API accepts tokens without verifying the signature, issuer, audience, expiration, or required claims, an attacker could present forged, expired, or incorrectly issued credentials. The API should use a trusted Keycloak issuer configuration, restrict accepted signing algorithms, validate all relevant claims, and reject tokens that are missing required identity or authorization attributes. Key rotation and key-discovery failures should produce a controlled authentication error rather than a permissive fallback.

Authorization failures represent a separate and potentially significant risk. A valid Keycloak token does not by itself prove that a user may access every requested object or administrative operation. Django views and service-layer functions should enforce role, group, scope, and object-level permissions on the server side. Resource identifiers supplied by clients must be checked against the authenticated principal’s permitted tenant, project, or ownership context. Client-side route restrictions in React should be treated only as usability controls, not security enforcement.

Authentication and session risks may arise from weak Keycloak client configuration, excessive token lifetimes, or insecure browser storage. Production clients should use an appropriate flow, avoid embedded client secrets in browser code, and request only necessary scopes. Access tokens should have limited lifetimes, while refresh-token handling should follow Keycloak’s rotation and revocation capabilities. Tokens should not be placed in URLs or exposed through application logs. Browser cookies, where used, should apply Secure, HttpOnly, and suitable SameSite attributes.

Database compromise could result from exposed PostgreSQL services, excessive application privileges, or unsafe query construction. PostgreSQL should be reachable only from authorized application networks, use encrypted connections where applicable, and employ separate roles with least-privilege permissions. Django’s parameterized ORM and query interfaces should be preferred, while raw SQL requires review. Secrets must be stored in a managed secret mechanism rather than source control, container images, or unprotected environment files.

Deployment and supply-chain weaknesses may allow unauthorized access or code execution. Containers and dependencies should be pinned, regularly updated, scanned, and built from controlled sources. Administrative interfaces for Keycloak, Django, and PostgreSQL should not be publicly exposed without a documented requirement and additional access controls. Network policies, reverse-proxy protections, rate limits, security headers, and centralized audit logging should be implemented to reduce attack impact and support detection. Backup files, logs, and error responses must also be reviewed for inadvertent disclosure of credentials, tokens, personal data, or internal system details.

11. Deployment and Release Checklist

Deployment readiness should be established through a documented release gate rather than assumed from successful local testing. The release owner should confirm that the intended build, configuration revision, database migration set, and container or package artifacts are uniquely identified and reproducible. Changes to application code, infrastructure, Keycloak configuration, and PostgreSQL schemas should be reviewed by appropriate technical owners, with security-impacting changes receiving explicit approval. The release record should identify the accountable owner, deployment window, rollback authority, and communication plan.

Before deployment, verify that production configuration is separated from development and test environments. Secrets, signing keys, database credentials, and external service credentials must be supplied through an approved secret-management mechanism rather than source control, container images, build logs, or client-side configuration. Confirm that the production frontend contains no administrative secrets and that API, Keycloak, database, and management endpoints use the intended hostnames, ports, transport security, and network access rules. Review cross-origin settings, redirect URIs, cookie attributes, token audiences and issuers, and environment-specific logging levels.

The release candidate should pass the project's defined automated checks, including dependency and container scanning, linting, unit and integration tests, authentication-flow tests, JWT validation tests, authorization tests, and database migration checks. Testing should include denied access cases, expired or malformed tokens, insufficient roles or scopes, unexpected issuer or audience values, and requests to resources belonging to another user or tenant where applicable. Results and known exceptions should be recorded; the absence of a test or an unverified control must not be represented as a confirmed security result.

Database preparation should include a reviewed migration plan, backup or recovery-point verification, compatibility checks between application and schema versions, and an assessment of migration duration and locking behavior. Where feasible, use backward-compatible changes that permit controlled rollback. Confirm that application service accounts retain only required database privileges and that destructive or irreversible operations require explicit authorization.

The deployment procedure should define sequencing, health checks, smoke tests, and stop conditions. A typical sequence includes infrastructure readiness, database changes, backend deployment, Keycloak or identity configuration updates, frontend publication, and post-deployment verification; the actual order must reflect compatibility constraints. Health checks should confirm application availability, database connectivity, authentication, token validation, authorization enforcement, and essential user journeys without exposing sensitive data.

After release, compare observed behavior with the expected configuration and monitor errors, authentication failures, authorization denials, latency, resource consumption, and database health. Preserve deployment logs and approval records. If critical checks fail, suspend further rollout and invoke the documented rollback or remediation plan. Close the release only after ownership of residual issues, follow-up actions, and operational handover has been recorded.

12. Monitoring, Logging, Incident Response, and Maintenance

Effective monitoring should provide sufficient visibility into authentication, authorization, application health, infrastructure status, and database activity without exposing tokens, passwords, personal data, or other sensitive content. The React frontend, Django API, Keycloak, PostgreSQL, reverse proxy, container platform, and operating system should produce structured logs with consistent timestamps, service identifiers, request or correlation identifiers, severity levels, and relevant outcome fields. Centralized collection should be used where practical so that logs remain available if an individual workload is compromised or unavailable.

The Django API should record authentication failures, rejected or expired JWTs, authorization denials, unusual request rates, validation failures, administrative actions, and significant configuration changes. Logs should identify the affected endpoint and result while excluding complete bearer tokens and unnecessary request-body data. Keycloak should provide visibility into login failures, account changes, client configuration changes, administrative activity, and abnormal authentication patterns. PostgreSQL logging should be configured selectively to support investigation of connection failures, privilege changes, schema changes, and other security-relevant events without creating excessive noise or disclosing sensitive query parameters.

Operational monitoring should include service availability, error rates, latency trends, resource utilization, database connection saturation, storage capacity, certificate expiry, backup status, and synchronization or integration failures. Alert thresholds should be defined according to the expected operating profile and reviewed after deployment; this assessment does not establish production baselines or confirmed alert values. Alerts should be routed to accountable responders, include enough context for triage, and distinguish transient service degradation from events requiring security investigation.

An incident response process should define preparation, detection, analysis, containment, eradication, recovery, and post-incident review. Initial triage should determine whether an event represents an operational fault, a suspected credential compromise, unauthorized access, data exposure, or malicious modification. Response procedures should include disabling or rotating affected credentials, revoking sessions or refresh tokens where supported, restricting compromised clients, preserving relevant logs, isolating affected services, and validating system integrity before restoration. Any decision to reset accounts, invalidate signing keys, or restore a database should consider service continuity and dependency impacts.

Logs and forensic records should be protected against unauthorized alteration through restricted access, centralized retention, time synchronization, and, where feasible, write-once or otherwise tamper-evident storage. Retention periods should reflect legal, contractual, operational, and investigative requirements rather than being assumed. Access to monitoring systems should use least privilege and be reviewed periodically.

Maintenance should include timely updates for Django, React dependencies, Keycloak, PostgreSQL, operating-system packages, container images, and supporting infrastructure. Scheduled vulnerability review, dependency scanning, backup restoration exercises, certificate rotation, access recertification, and review of monitoring rules should be incorporated into normal operations. Changes to security configuration should be peer-reviewed, documented, tested in a representative environment, and reversible through a defined rollback procedure.

13. Final Recommendations and Technical Review Priorities

Project Aurora-927 should proceed only through a controlled deployment process that preserves clear trust boundaries between the React client, Django API, Keycloak identity provider, PostgreSQL database, and supporting infrastructure. The principal recommendation is to treat authentication, authorization, secret management, and operational observability as a single security control system rather than as independent implementation tasks. Final approval should be based on documented configuration review and evidence from the target environment, not on assumptions derived from the reference architecture.

The highest priority is to complete a technical review of JWT handling and authorization enforcement. The review should confirm that the Django API validates token signatures against trusted Keycloak keys, enforces expected issuer and audience values, checks expiration and relevant time claims, and rejects unsupported algorithms or malformed tokens. Authorization decisions should be enforced server-side for every protected endpoint, with roles, scopes, or claims mapped to explicit business permissions. Client-side route protection may improve usability but must not be treated as an access-control boundary.

Deployment review should next verify the separation of public, application, identity, and database network zones. PostgreSQL should accept connections only from approved application components, use encrypted connections where appropriate, and operate with least-privilege roles that separate application activity from administrative functions. Production secrets, signing-related configuration, database credentials, and service tokens should be supplied through an approved secret-management process rather than source control, frontend bundles, or unprotected environment files. Backup, restoration, and key-rotation procedures should be documented and periodically exercised.

Before release, reviewers should examine Django security settings, dependency versions, administrative interfaces, error handling, CORS and CSRF configuration, rate limiting, and production logging. Logs should support investigation of authentication failures, denied authorization decisions, administrative actions, configuration changes, and unusual API activity without recording passwords, tokens, or unnecessary personal data. Alert thresholds and escalation ownership should be defined before production exposure.

The remaining technical review should prioritize evidence collection: deployment manifests, firewall or security-group rules, Keycloak client and realm settings, API permission mappings, database grants, TLS configuration, secret-rotation records, backup policies, and monitoring dashboards. Each item should identify an owner, review date, and disposition for unresolved issues. The eight risks described in this report are hypothetical assessment concerns unless independently confirmed; they should therefore be converted into verification tasks rather than reported as established vulnerabilities.

Finally, Aurora-927 should adopt a recurring security maintenance cycle covering dependency updates, access reviews, configuration drift, log-retention requirements, incident-response exercises, and reassessment after material architectural changes. This approach provides a defensible basis for release approval while maintaining security assurance throughout the system’s operational life.

14. Appendices: Glossary, Assessment Basis, and Page Numbering

Glossary

Access token: A short-lived token presented by a client to request access to protected API resources. In this assessment, access tokens are assumed to use the JSON Web Token (JWT) format.

Authentication: The process of verifying the identity of a user, service, or other client. Keycloak is treated as the identity and access-management component responsible for issuing or brokering authenticated sessions and tokens.

Authorization: The process of determining whether an authenticated principal may perform a requested action or access a specific resource. Authorization decisions should be enforced by the Django API rather than inferred solely from client-side behavior.

Claim: A named item of information contained in a JWT, such as the subject, issuer, audience, expiry time, or assigned roles.

Django API: The server-side application component that processes requests, applies authentication and authorization controls, and communicates with PostgreSQL.

Keycloak: The identity and access-management platform assumed to issue, validate, and administer application identities, clients, realms, roles, and related authentication policies.

PostgreSQL: The relational database assumed to store application data. It should be reachable only through controlled application paths and protected with least-privilege database accounts.

Refresh token: A credential used to obtain a replacement access token. Its longer validity and sensitivity require stricter storage, rotation, revocation, and exposure controls than ordinary access tokens.

Trust boundary: A point where data, credentials, or requests cross between components with different security assumptions, such as between a browser, reverse proxy, API, identity provider, and database.

Assessment Basis

This report is a design and deployment assessment for the fictional Project Aurora-927 application. Its basis is the stated technology context—React, Django, Keycloak, and PostgreSQL—together with the security concerns identified in the report outline. The analysis considers expected control objectives, common deployment patterns, authentication and authorization dependencies, data-flow boundaries, operational practices, and plausible failure modes.

The risks described are hypothetical risks or control gaps requiring confirmation; they are not statements that vulnerabilities were observed or exploited. No source code, configuration export, infrastructure inventory, penetration-test evidence, runtime logs, database inspection, or independently verified test results was provided for this assessment. Accordingly, recommendations should be validated during implementation and tested against the actual deployment, including token issuer and audience settings, key rotation behavior, role mapping, network exposure, database privileges, secret storage, logging, and recovery procedures.

Where the report uses terms such as “should,” “recommended,” or “assumed,” the wording identifies a proposed control or assessment premise rather than a confirmed implementation fact. A formal security review should retain evidence for each completed control, record exceptions, and distinguish accepted residual risk from unresolved remediation work.

Page Numbering

The final PDF should apply sequential page numbers consistently across the document, preferably in the footer with a format such as “Page X of Y.” Numbering should begin with the first report page or, if a cover page is included, follow the organization’s document-control convention and make that convention explicit. The table of contents, if present, should correspond to the final rendered pagination.

Page numbers should be generated after layout is finalized so that page breaks, tables, headings, and appendices are accurately reflected. Any exported or revised version should be checked for missing, duplicated, or misaligned numbers, particularly on pages containing wide risk tables or section transitions.